

LECTURE 5

SQL FUNCTIONS.

Objective: To develop students' systematic knowledge regarding the use of SQL functions.

Key topics

1. Basic concepts
2. Single-row functions
3. Multi-row functions
4. Window functions

1. BASIC CONCEPTS

Creating User-Defined Functions (UDFs) in SQL depends on the specific DBMS being used (e.g., PostgreSQL, MySQL, or MS SQL Server), but the fundamental approach is similar across systems: the function accepts parameters, executes logic, and returns a result (either a single value or a table).

SQL functions execute a sequence of SQL statements and return the result of the final query in the list. In a simple case (where a set is not returned), the first row of the final query's result is returned. (Keep in mind that the concept of the "first row" in a multi-row result set is only precisely defined if an `ORDER BY` clause is present.) If the final query returns no rows at all, a `NULL` value is returned.

Additionally, an SQL function can be declared to return a set (i.e., multiple rows) by specifying `SETOF some\_type` as the return type or by declaring it with `RETURNS TABLE(columns)`. In this case, all rows from the result of the final query are returned.

The body of an SQL function must be a list of SQL statements separated by semicolons. A semicolon after the last statement is optional. Unless the function is declared to return `void`, the final statement must be a `SELECT`, or an `INSERT`, `UPDATE`, `DELETE`, or `MERGE` statement with a `RETURNING` clause.

Any set of SQL commands can be grouped together and defined as a function. In addition to `SELECT` queries, these commands can include data-modifying statements (`INSERT`, `UPDATE`, `DELETE`, and `MERGE`) as well as other SQL commands. (Transaction control commands - such as `COMMIT` or `SAVEPOINT` - and certain utility commands, notably `VACUUM`, cannot be used in SQL functions.) However, the final command must be a `SELECT` or a statement with a `RETURNING` clause that returns a result matching the function's return type. If you wish to define an SQL function that performs actions but does not return a useful value, you can declare it to return `void`. For example, this function deletes rows with negative salaries from the `emp` table:

PostgreSQL Example

```
CREATE FUNCTION clean_emp() RETURNS void AS '  
    DELETE FROM emp  
        WHERE salary < 0;  
' LANGUAGE SQL;  
  
SELECT clean_emp();  
  
clean_emp  
-----  
  
(1 row)
```

If you write the same thing as a procedure, you do not need to specify a return type. For example:

PostgreSQL example

```
CREATE PROCEDURE clean_emp() AS '  
    DELETE FROM emp  
        WHERE salary < 0;  
' LANGUAGE SQL;  
  
CALL clean_emp();
```

In such simple cases, the difference between a procedure and a function returning `void` is primarily stylistic. However, compared to functions, procedures offer additional functionality, such as transaction control capabilities. Furthermore, procedures comply with the SQL standard, whereas functions returning `void` are specific to Postgres Pro.

The syntax for the `CREATE FUNCTION` command requires the function body to be written as a string constant. Typically, the most convenient way to do this is to enclose the string constant in `\$` delimiters.

Let us look at creating the main types of functions.

1. Scalar functions (return a single value)

A scalar function returns a single value of a specific data type (number, string, date, etc.).

Example in MS SQL Server (T-SQL):

sql

```
CREATE FUNCTION dbo.CalculateTotal (@Price DECIMAL(10,2), @Qty INT)
RETURNS DECIMAL(10,2)
AS
BEGIN
    RETURN @Price * @Qty;
END;
GO

-- Function call (specifying the dbo schema is mandatory):
SELECT dbo.CalculateTotal(150.00, 3);
```

2. Table-valued functions (return a set of rows)

A table-valued function returns a result in the form of a table, allowing it to be used in FROM or JOIN clauses just like a regular table or view.

Example in MS SQL Server:

```
CREATE FUNCTION dbo.GetProductsByPrice (@MinPrice DECIMAL(10,2))
RETURNS TABLE
AS
RETURN
    SELECT ProductID, ProductName, Price
    From Products
    WHERE Price >= @MinPrice;
GO

-- Function call:
SELECT * FROM dbo.GetProductsByPrice(500.00);
```

Key differences between a function and a stored procedure:

- **Invocation:** A function can be called directly within SQL queries (SELECT, WHERE, JOIN), whereas a procedure is called via a separate EXEC (or CALL) command.
- **Data modification:** A function must not alter the database state (it cannot perform INSERT, UPDATE, or DELETE operations on permanent tables). Procedures support all types of data modification and transaction management (BEGIN TRAN, COMMIT).

Functions used in SQL queries can be divided into two broad categories:

- functions that return a single value based on the data in a single row (single-row functions).
- functions that return a single value based on multiple rows of data (multi-row functions).

Any SQL function - whether single-row or multi-row - is designed to accept parameters (or none, in the case of parameterless functions) and return exactly one value of a specific type.

A brief clarification: regarding the "single value" returned by a function, we should not overlook the fact that SQL functions can have OUT parameters, allowing them to return multiple values. Here,

we are referring to the value returned directly by the function itself, rather than values returned via parameters.

We will focus on SQL functions that can be used in a SELECT statement, noting that functions with OUT parameters cannot be used in this context.

2. SINGLE-ROW FUNCTIONS

Single-row functions in SQL are functions that process data from a single row and return exactly one result for each row processed.

Unlike aggregate functions (SUM, COUNT, AVG), they do not combine or “collapse” several rows into one, but maintain the original number of records in the table.

Main features:

- **Scope:** Apply to each line separately.
- **Where used:** In SELECT, WHERE, ORDER BY statements and other parts of the query.
- **Nesting:** They can be nested inside each other (for example, apply UPPER to the result of the SUBSTR function)

Main Categories of Single-row Functions

- **String (character):** Modify or parse texts (for example, LOWER, UPPER to change case, SUBSTR to extract part of a string, LENGTH to count length).
- **Numeric:** Perform mathematical operations on numbers (for example, ROUND for rounding, ABS for modulus, MOD for remainder).
- **Datetime:** Works with calendar and timestamps (for example, SYSDATE or CURRENT_DATE for the current date, ADD_MONTHS for adding months).
- **Conversion functions:** Change data types (for example, CAST or TO_CHAR convert numbers/dates to text format).
- **Dealing with NULL:** Helps handle empty values (for example, COALESCE or NVL return a fallback value if NULL is encountered)

CHARACTER FUNCTIONS

SQL string functions are used to search, modify, concatenate, and process text data.

Key categories and functions

- **String length:**

LENGTH() or LEN() - returns the number of characters in the string.

```
SELECT LEN('Apple') -- 5
```

- **Change case:**

LOWER() - converts all characters in the string to lowercase.

UPPER() - converts all characters in the string to uppercase.

```
SELECT LOWER('Apple') -- apple
```

```
SELECT UPPER('Apple') -- APPLE
```

- **Joining Strings (Concatenation):**

CONCAT() - combines two or more strings into one.

The || operator (in PostgreSQL, Oracle, SQLite) or + (in MS SQL Server) for string concatenation..

```
SELECT CONCAT('Tom', ' ', 'Smith') -- Tom Smith
```

- **Substring extraction:**

SUBSTRING() or SUBSTR() - extracts a part of a string based on a specified index and length.

LEFT() / RIGHT() - returns the specified number of characters from the beginning or end of the string.

```

SELECT LEFT('Apple', 3) - App
SELECT RIGHT('Apple', 3)      -- ple
SELECT SUBSTRING('Galaxy S8 Plus', 8, 2)      -- S8

```

– **Removing spaces:**

TRIM() - removes whitespace (or specified characters) from both ends of the string.

LTRIM() / RTRIM() - removes spaces only from the left or right edge.

```

SELECT LTRIM('  Apple')
SELECT RTRIM(' Apple  ')

```

– **Find and replace:**

REPLACE() - replaces one occurrence of a substring with another.

REVERSE() - reverses the string.

POSITION() or CHARINDEX() - finds the position of the first occurrence of a substring in the text.

PATINDEX() - returns the index of the first occurrence of a specific pattern in the string.

SPACE() - returns a string containing a specified number of spaces.

```

SELECT REPLACE('Galaxy S8 Plus', 'S8 Plus', 'Note 8')      -- Galaxy Note 8
SELECT REVERSE('123456789') -- 987654321

SELECT CHARINDEX('pl', 'Apple') - 3

SELECT PATINDEX('%p_e%', 'Apple')      -- 3

```

For example, let's take a table:

```

CREATE TABLE Products
(
    Id INT IDENTITY PRIMARY KEY,
    ProductName NVARCHAR(30) NOT NULL,
    Manufacturer NVARCHAR(20) NOT NULL,
    ProductCount INT DEFAULT 0,
    Price MONEY NOT NULL
);

```

And we will apply string functions when retrieving the data:

```

SELECT UPPER(LEFT(Manufacturer,2)) AS Abbreviation,
       CONCAT(ProductName, ' - ', Manufacturer) AS FullProdName
FROM Products
ORDER BY Abbreviation

```

1	USE productsdb;
2	
3	SELECT UPPER(LEFT(Manufacturer,2)) AS Abbreviation,
4	CONCAT(ProductName, ' - ', Manufacturer) AS FullProdName
5	FROM Products
6	ORDER BY Abbreviation

	Abbreviation	FullProdName
1	AP	iPhone 6 - Apple
2	AP	iPhone 6S - Apple
3	AP	iPhone 7 - Apple
4	ON	OnePlus 5 - OnePlus
5	SA	Galaxy S8 - Samsung
6	SA	Galaxy S8 Plus - Samsung
7	XI	Mi 5X - Xiaomi

DATETIME FUNCTIONS

SQL provides a number of functions for working with dates and times:

- **GETDATE:** returns the current local date and time based on the system clock as a datetime object

```
SELECT GETDATE ()      -- 2017-07-28 21:34:55.830
```

- **GETUTCDATE:** returns the current local date and time in Greenwich Mean Time (UTC/GMT) as a datetime object

```
SELECT GETUTCDATE ()   -- 2017-07-28 18:34:55.830
```

- **SYSDATETIME:** returns the current local date and time based on the system clock, but unlike GETDATE, the date and time are returned as a datetime2 object

```
SELECT SYSDATETIME ()  -- 2017-07-28 21:02:22.7446744
```

- **SYSUTCDATETIME:** returns the current local date and time in Greenwich Mean Time (UTC/GMT) as a datetime2 object

```
SELECT SYSUTCDATETIME ()  -- 2017-07-28 18:20:27.5202777
```

- **SYSDATETIMEOFFSET:** returns a datetimeoffset(7) object containing the date and time relative to GMT

```
SELECT SYSDATETIMEOFFSET ()  -- 2017-07-28 21:02:22.7446744 +03:00
```

- **DAY:** returns the day of the date passed as a parameter

```
SELECT DAY (GETDATE ())  -- 28
```

- **MONTH:** returns the month of the date

```
SELECT MONTH (GETDATE ())  -- 7
```

- **YEAR:** returns the year of the date

```
SELECT YEAR (GETDATE ())      -- 2017
```

– **DATENAME:** returns a part of the date as a string. The parameter specifying the date part is passed as the first argument, and the date itself is passed as the second argument.

```
SELECT DATENAME (month, GETDATE ())      -- July
```

– To specify a part of a date, you can use the following parameters (abbreviated versions are shown in parentheses):

- year (yy, yyyy)
- quarter (qq, q)
- month (mm, m)
- dayofyear (dy, y)
- day (dd, d)
- week (wk, ww)
- weekday (dw)
- hour (hh)
- minute (mi, n)
- second (ss, s)
- millisecond (ms)
- microsecond (mcs)
- nanosecond (ns)
- tzoffset (tz): offset in minutes relative to GMT (for a datetimeoffset object)

– **DATEPART:** returns a part of the date as a number. The parameter specifying the date part is passed as the first argument (using the same parameters as for DATENAME), and the date itself is passed as the second argument:

```
SELECT DATEPART (month, GETDATE ())      -- 7
```

– **DATEADD:** returns a date resulting from adding a number to a specific date component. The first parameter represents the date component described above for the DATENAME function. The second parameter is the amount to be added. The third parameter is the date itself to which the addition is to be made.

```
SELECT DATEADD (month, 2, '2017-7-28')    -- 2017-09-28 00:00:00.000
SELECT DATEADD (day, 5, '2017-7-28')      -- 2017-08-02 00:00:00.000
SELECT DATEADD (day, -5, '2017-7-28')     -- 2017-07-23 00:00:00.000
```

If the added amount is a negative number, the date is effectively decreased.

– **DATEDIFF:** returns the difference between two dates. The first parameter is a date component specifying the units in which to measure the difference. The second and third parameters are the dates being compared:

```
SELECT DATEDIFF (year, '2017-7-28', '2018-9-28')    -- разница 1 год
SELECT DATEDIFF (month, '2017-7-28', '2018-9-28')   -- разница 14 месяцев
SELECT DATEDIFF (day, '2017-7-28', '2018-9-28')     -- разница 427 дней
```

– **TODATETIMEOFFSET:** returns a datetimeoffset value that is the result of adding a time offset to a datetime2 object

```
SELECT TODATETIMEOFFSET ('2017-7-28 01:10:22', '+03:00')
```

– **SWITCHOFFSET:** returns a datetimeoffset value that is the result of adding a time offset to another datetimeoffset object

```
SELECT SWITCHOFFSET (SYSDATETIMEOFFSET (), '+02:30')
```

– **EOMONTH:** returns the date of the last day of the month used in the date passed as a parameter

```
SELECT EOMONTH ('2017-02-05')      -- 2017-02-28
SELECT EOMONTH ('2017-02-05', 3)  -- 2017-05-31
```

You can pass the number of months to add to the date as an optional second parameter. In this case, the last day of the month will be calculated based on the new date.

– **DATEFROMPARTS:** creates a date from the year, month, and day

```
SELECT DATEFROMPARTS (2017, 7, 28)      -- 2017-07-28
```

– **ISDATE:** checks whether the expression is a date. If it is, it returns 1; otherwise, it returns 0.

```
SELECT ISDATE ('2017-07-28')      -- 1
SELECT ISDATE ('2017-28-07')      -- 0
SELECT ISDATE ('28-07-2017')      -- 0
SELECT ISDATE ('SQL')             -- 0
```

An example of using functions is the creation of an orders table that includes the order date:

```
CREATE TABLE Orders
(
    Id INT IDENTITY PRIMARY KEY,
    ProductId INT NOT NULL,
    CustomerId INT NOT NULL,
    CreatedAt DATE NOT NULL DEFAULT GETDATE(),
    ProductCount INT DEFAULT 1,
    Price MONEY NOT NULL
);
```

The expression `DEFAULT GETDATE()` specifies that if no date is provided when adding data, it is automatically calculated using the `GETDATE()` function.

As another example, let's find orders placed 16 days ago:

```
SELECT * FROM Orders
WHERE DATEDIFF(day, CreatedAt, GETDATE()) = 16
```

The screenshot shows a SQL query window with the following text:

```

1 USE productsdb;
2
3 SELECT * FROM Orders
4 WHERE DATEDIFF(day, CreatedAt, GETDATE()) = 16

```

Below the query window is a results grid with the following data:

	Id	ProductId	CustomerId	CreatedAt	ProductCount	Price
1	2	2	1	2017-07-13	1	41000.00

NUMERIC FUNCTIONS

SQL provides a number of functions for working with numeric data:

- **ROUND:** Rounds a number. The first parameter is the number itself. The second parameter specifies the length. If the length is a positive number, it indicates the number of decimal places to which rounding occurs. If the length is a negative number, it indicates the position relative to the decimal point, counting backwards from the end of the number.

```

SELECT ROUND(1342.345, 2)    -- 1342.350
SELECT ROUND(1342.345, -2)   -- 1300.000

```

- **ISNUMERIC:** Determines whether a value is a number. The function accepts an expression as a parameter. If the expression is a number, the function returns 1; otherwise, it returns 0.

```

SELECT ISNUMERIC(1342.345)    -- 1
SELECT ISNUMERIC('1342.345')  -- 1
SELECT ISNUMERIC('SQL')       -- 0
SELECT ISNUMERIC('13-04-2017') -- 0

```

- **ABS:** returns the absolute value of a number

```

SELECT ABS(-123)    -- 123

```

- **CEILING:** returns the smallest integer greater than or equal to the current value

```

SELECT CEILING(-123.45)    -- -123
SELECT CEILING(123.45)     -- 124

```

- **FLOOR:** returns the largest integer less than or equal to the current value

```

SELECT FLOOR(-123.45)     -- -124
SELECT FLOOR(123.45)      -- 123

```

- **SQUARE:** squares the number

```

SELECT SQUARE(5)          -- 25

```

- **SQRT:** calculates the square root of a number

```

SELECT SQRT(225)          -- 15

```

- **RAND:** generates a random floating-point number in the range from 0 to 1.

```
SELECT RAND()          -- 0.707365088352935
SELECT RAND()          -- 0.173808327956812
```

For example, let's take a table:

```
CREATE TABLE Products
(
    Id INT IDENTITY PRIMARY KEY,
    ProductName NVARCHAR(30) NOT NULL,
    Manufacturer NVARCHAR(20) NOT NULL,
    ProductCount INT DEFAULT 0,
    Price MONEY NOT NULL
);
```

Let us round the product of the product's price and the quantity of that product:

```
SELECT ProductName, ROUND(Price * ProductCount, 2)
FROM Products
```

NEWID, ISNULL II COALESCE FUNCTIONS

NEWID

The NEWID() function is used to generate a UNIQUEIDENTIFIER object – that is, a unique value. For example, we can define the primary key column as type UNIQUEIDENTIFIER and assign it the value of the NEWID function by default:

```
CREATE TABLE Clients
(
    Id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
    FirstName NVARCHAR(20) NOT NULL,
    LastName NVARCHAR(20) NOT NULL,
    Phone NVARCHAR(20) NULL,
    Email NVARCHAR(20) NULL
)

INSERT INTO Clients (FirstName, LastName, Phone, Email)
VALUES ('Tom', 'Smith', '+36436734', NULL),
('Bob', 'Simpson', NULL, NULL)
```

ISNULL

The ISNULL function checks the value of an expression. If the value is NULL, the function returns the value passed as the second parameter:

```
ISNULL(expression, value)
```

For example, let's take the table created earlier and apply the ISNULL function when retrieving data:

```
SELECT FirstName, LastName,
    ISNULL(Phone, 'undefined') AS Phone,
    ISNULL(Email, 'unknown') AS Email
FROM Clients
```

```

1 USE productsdb;
2
3 SELECT FirstName, LastName,
4        ISNULL(Phone, 'неопределено') AS Phone,
5        ISNULL(Email, 'неизвестно') AS Email
6 FROM Clients

```

	FirstName	LastName	Phone	Email
1	Bob	Simpson	неопределено	неизвестно
2	Tom	Smith	+36436734	неизвестно

COALESCE

The COALESCE function takes a list of values and returns the first one that is not NULL:

```
COALESCE(expression_1, expression_2, ... expression_N)
```

For example, let's select users from the `Clients` table and identify either a phone number or an email address from their contact details, provided these values are not NULL:

```

SELECT FirstName, LastName,
       COALESCE(Phone, Email, 'undefined') AS Contacts
FROM Clients

```

In this case, the phone number is returned if it is defined. If it is not defined, the email address is returned. If the email address is also not defined, the string "undefined" is returned.

```

1 USE productsdb;
2
3 SELECT FirstName, LastName,
4        COALESCE(Phone, Email, 'неопределено') AS Contacts
5 FROM Clients

```

	FirstName	LastName	Contacts
1	Bob	Simpson	неопределено
2	Tom	Smith	+36436734

3. MULTI-LINE FUNCTIONS

Multi-row functions in SQL are aggregate functions that process a set of rows (multiple rows) and return a single value for the entire group or for data divided into groups.

Key aggregate functions

- **COUNT()**: counts the number of rows or non-empty values in a column.
- **SUM()**: calculates the total sum of a numeric column.
- **AVG()**: calculates the arithmetic mean of a numeric column.
- **MIN()**: determines the minimum value in a set.
- **MAX()**: determines the maximum value in a set.

Key features

- Row reduction: unlike single-row (scalar) functions, which operate on each row individually, multi-row functions condense multiple rows into a single one.
- Use with GROUP BY: they are often used in conjunction with the grouping operator to calculate metrics for each category separately.
- Ignoring NULLs: all standard aggregate functions (except COUNT(*)) skip null values (NULL) during calculations.

Usage example

Calculate the total amount and average price of products for each category:

```
SELECT category_id,  
       SUM(price) AS total_sum,  
       AVG(price) AS average_price  
FROM products  
GROUP BY category_id;
```

The screenshot shows a SQL query editor with the following query in the 'Input' tab:

```
SELECT  
  COUNT(*) AS total_customers,  
  COUNT(country) AS customers_with_country,  
  COUNT(DISTINCT country) AS unique_countries  
FROM Customers;
```

The 'Output' tab displays the results of the query:

total_customers	customers_with_country	unique_countries
5	5	3

The 'Available Tables' section on the right shows the structure of the 'Customers' and 'Orders' tables.

Customers

customer_id	first_name	last_name	age	country
1	John	Doe	31	USA
2	Robert	Luna	22	USA
3	David	Robinson	22	UK
4	John	Reinhardt	25	UK
5	Betty	Doe	28	UAE

Orders

order_id	item	amount	customer_id
1	Keyboard	400	4
2	Mouse	300	4
3	Monitor	12000	3
4	Keyboard	400	1
5	Mousepad	250	2

The screenshot shows a SQL query editor with the following query in the 'Input' tab:

```
SELECT  
  SUM(amount)  
FROM Orders;
```

The 'Output' tab displays the results of the query:

SUM(amount)
13350

The 'Available Tables' section on the right shows the structure of the 'Orders' and 'Shippings' tables.

Orders

order_id	item	amount	customer_id
1	Keyboard	400	4
2	Mouse	300	4
3	Monitor	12000	3
4	Keyboard	400	1
5	Mousepad	250	2

Shippings

shipping_id	status	customer
1	Pending	2

The screenshot shows a SQL query editor with the following query in the 'Input' tab:

```
SELECT MIN(age) AS min_age,  
       MAX(age) AS max_age  
FROM Customers;
```

The 'Output' tab displays the results of the query:

min_age	max_age
22	31

The 'Available Tables' section on the right shows the structure of the 'Customers' and 'Orders' tables.

Customers

customer_id	first_name	last_name	age	country
1	John	Doe	31	USA
2	Robert	Luna	22	USA
3	David	Robinson	22	UK
4	John	Reinhardt	25	UK
5	Betty	Doe	28	UAE

Orders

order_id	item	amount	customer_id
1	Keyboard	400	4
2	Mouse	300	4
3	Monitor	12000	3
4	Keyboard	400	1
5	Mousepad	250	2

Input

```
SELECT customer_id, SUM(amount)
FROM Orders
WHERE amount > 200
GROUP BY customer_id
HAVING SUM(amount) > 1000;
```

Output

customer_id	SUM(amount)
3	12000

Available Tables

Customers

customer_id	first_name	last_name	age	country
1	John	Doe	31	USA
2	Robert	Luna	22	USA
3	David	Robinson	22	UK
4	John	Reinhardt	25	UK
5	Betty	Doe	28	UAE

Orders

order_id	item	amount	customer_id
1	Keyboard	400	4
2	Mouse	300	4
3	Monitor	12000	3
4	Keyboard	400	1
5	Mousepad	250	2

Common mistakes when working with aggregate functions

Let's look at the main SQL errors that frequently arise when designing complex reports:

Using an aggregate function in the WHERE clause. As discussed earlier, a condition like `WHERE COUNT(id) > 5` will break the query. Use only `HAVING` to filter groups that have already been formed.

Ungrouped columns in the SELECT clause. If you have grouped data by `client\_id`, you cannot simply include the `order\_date` column in the output. A single client may have multiple orders placed on different days, and the database won't know which specific date to display in the result row. Any column in the SELECT clause must either be listed in the `GROUP BY` clause or wrapped in an aggregate function (e.g., `MAX(order\_date)` to show the date of the most recent order).

Confusing `COUNT(\*)` with `COUNT(column)`. Calling `COUNT(phone\_number)` will return the number of users who provided a phone number in their profile. To find the actual number of registered users – including those with a `NULL` phone number – you should always use `COUNT(\*)`.

4. WINDOW FUNCTIONS

Window functions are a powerful SQL tool that enables complex calculations across groups of rows related to the current row.

How they work

You might be wondering: "What does 'window' mean?"

In a standard SQL query, the entire set of rows is treated as a single, continuous block of data for which aggregate values are calculated.

However, when window functions are used, a group of related rows (or a "window") is defined for each row, and an individual aggregate value is calculated for that group.

This set of rows can be:

- the entire table
- individual table partitions - that is, a group of rows based on one or more fields
- or a window frame - a specific range of rows within the table or partition. For example, we can define a window frame to be passed to a window function consisting of the previous row and the current row of the table. In this case, the aggregate function value for each row will be calculated differently, as the data fed into the function changes dynamically from row to row. The frame effectively "slides" across the table.

Window function syntax

```
SELECT <window function> (<field_table>)
OVER (
  [PARTITION BY <columns_to_split>]
  [ORDER BY <columns_to_sort>]
```

) [ROWS|RANGE <row_range_definition>]

Where\

\:

– <window function> (<field_table>) - the window function used. For example, AVG(price).

– Next comes the OVER clause, which defines the window (group of rows) passed to the window function. If the OVER() construct is used without parameters, the window encompasses all rows of the query result.

– Inside the OVER clause, there are three optional parameters that allow for flexible window configuration:

- PARTITION BY <partitioning_columns> divides the result set into non-overlapping subsets – partitions – where each subset contains rows sharing the same values in one or more columns.
- ORDER BY <sorting_columns> establishes the row order within the window; this plays a particularly important role in ranking window functions.
- ROWS|RANGE <row_range_definition> defines the window frame. This parameter allows you to specify how many rows to include before and after the current row.

Quiz questions:

1. What are single-row functions in SQL? Describe the main categories of single-row functions and provide examples of their use in SQL queries.
2. Explain the purpose of string and numeric single-row SQL functions. Provide an SQL query that uses at least two such functions simultaneously to process table data.
3. What are multi-row (aggregate) SQL functions? Describe the purpose of the COUNT(), SUM(), AVG(), MIN(), and MAX() functions, and provide an example of a query that groups data using the GROUP BY clause.
4. Explain the difference between the WHERE and HAVING clauses when using multi-row functions. Provide an SQL query that groups records, calculates an aggregate value, and selects only those groups that meet a specific condition.
5. What are SQL window functions, and how do they differ from multi-row (aggregate) functions? Explain the purpose of the OVER() clause and provide an example query using a window function, such as ROW_NUMBER(), RANK(), or SUM() OVER().
6. Using the same table as an example, explain the difference between using an aggregate function with GROUP BY and a window function with OVER (PARTITION BY ...). Provide two SQL queries and explain how the result of each query differs.